PROGRAMACIÓN II

Grado en Ingeniería en Inteligencia Artificial
Universidad de Alicante

T01

PRESENTACIÓN Y REPASO DE PROGRAMACIÓN I

PROFESORADO

- **Miguel A. Teruel** – materuel@dlsi.ua.es
  - Teoría (Viernes 11:00 – 13:00)
  - Práctica (Viernes 9:00 – 11:00)
  - Práctica (Viernes 13:00 – 15:00)

- **Cristina Cachero** – ccc@ua.es
  - Práctica (Lunes 13:00 – 15:00)

Teoría

Tomar apuntes, probar código
y hacer ejercicios

Práctica

Ejercicios con ordenador y
trabajo individual / grupal

PLANIFICACIÓN CLASES TEÓRICAS

Semana	Contenidos
1	Repaso
2	Programación Orientada a Objetos: Clases (I)
3	Programación Orientada a Objetos: Clases (II)
4	Programación Orientada a Objetos: Herencia (I)
5	Programación Orientada a Objetos: Herencia (II)
6	Programación Orientada a Objetos: Sobrecarga del Operador (I)
7	Programación Orientada a Objetos: Sobrecarga del Operador (II)
8	Excepciones
9	Ficheros de Texto
10	Ficheros Binarios
11	Programación Avanzada en Python (I)
12	Programación Avanzada en Python (II)
13	Programación Orientada a Objetos Avanzada (I)
14	Programación Orientada a Objetos Avanzada (II)

PLANIFICACIÓN CLASES PRÁCTICAS

Semana	Contenidos
1	Linux + VENV
2	Presentación Trabajo Final + PyCharm + Repaso Programación I
3	GIT (I)
4	GIT (II)
5	Documentación de Código + Ejercicios Orientación a Objetos (I)
6	Ejercicios Orientación a Objetos (II)
7	Ejercicio de autoevaluación de Orientación a Objetos
8	Examen Tipo Test
9	Evaluación cruzada de las prácticas (solo Orientación a Objetos)
10	API RESTful con Python Flask
11	Despliegue de aplicaciones Python en la Nube
12	Acceso a Bases de Datos en Python
13	Simulacro de Examen Final
14	Defensa de Prácticas

PRUEBAS DE EVALUACIÓN

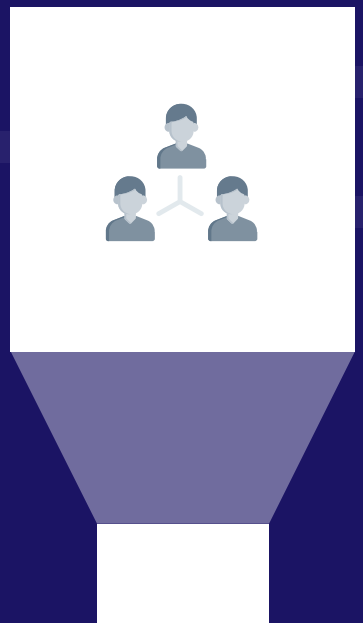
Parcial

Tipo Test

Prácticas

Trabajo en grupos de 5 o 6 personas

Nota individual

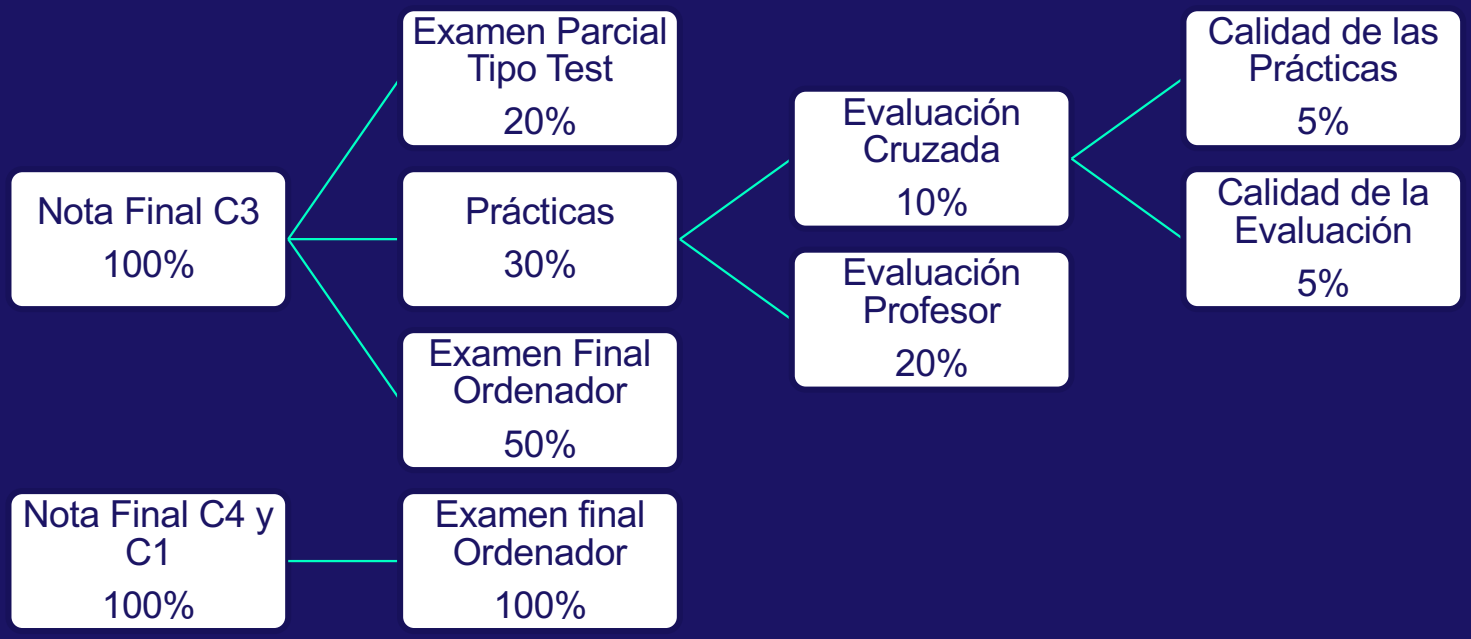


Final

Ordenador Offline

Cheatsheet a mano

EVALUACIÓN


Nota Final C3
100%
Examen Parcial Tipo Test
20%
Prácticas
30%
Examen Final Ordenador
50%
Evaluación Cruzada
10%
Evaluación Profesor
20%
Calidad de las Prácticas
5%
Calidad de la Evaluación
5%
Nota Final C4 y C1
100%
Examen final Ordenador
100%

COPIAS Y PLAGIOS

Para todas las convocatorias y exámenes, la detección de copia o plagio, o intento del mismo, supondrá la **calificación de "0"** en el examen correspondiente. Además, se informará a la dirección de Departamento y al Centro sobre esta incidencia para que se tomen las medidas pertinentes. La reiteración de esta conducta conllevará a la notificación al vicerrectorado correspondiente de las faltas cometidas para que estudien el caso y sancionen según la legislación (Reglamento de disciplina académica de los Centros oficiales de Enseñanza Superior y de Enseñanza Técnica dependientes del Ministerio de Educación Nacional BOE 12/10/1954).

Queda terminante prohibido hacer fotos tanto al proyector como a la pizarra

Pero puedes (o mejor dicho debes) copiar lo que quieras ...

BIBLIOGRAFÍA

- Luciano, R. (2015). Fluent Python: Clear, concise, and effective programming. O'Reilly Media, Inc. 2nd edition
- Matthes, E. (2023). Python crash course: A hands-on, project-based introduction to programming. no starch press.
- van Hattem, R. (2022). Mastering Python: Write Powerful and Efficient Code Using the Full Range of Python's Capabilities. Packt Publishing Limited.
- Lott, S. F., & Phillips, D. (2021). Python Object-Oriented Programming: Build robust and maintainable object-oriented Python applications and libraries. Packt Publishing Ltd.
- Python Software Foundation (2023) Python Documentation (https://www.python.org/doc/)
- Real Python (2023) Learn Python (https://realpython.com)
- W3Schools (2023) Python Tutorial (https://www.w3schools.com/python/default.asp)
- Alfredo Sánchez Alberca (2023) Aprende con Alf - Recursos Educativos Libres (https://aprendeconalf.es)

REPASO DE PROGRAMACIÓN I

OPERADORES MATEMÁTICOS

De mayor a menor precedencia:

Operador	Operación	Ejemplo
**	Exponente	2 ** 3 = 8
%	Módulo / resto	22 % 8 = 6
//	División entera	22 // 8 = 2
/	División	22 / 8 = 2.75
*	Multiplicación	3 * 3 = 9
-	Resta	5 - 2 = 3
+	Suma	2 + 2 = 4

EJERCICIO

```
>>> 2 + 3 * 6

>>> (2 + 3) * 6

>>> 2 ** 8

>>> 23 // 7

>>> 23 % 7

>>> (5 - 1) * ((7 + 1) / (3 - 1))
```

SOLUCIÓN

```
>>> 2 + 3 * 6
# 20

>>> (2 + 3) * 6
# 30

>>> 2 ** 8
#256

>>> 23 // 7
# 3

>>> 23 % 7
# 2

>>> (5 - 1) * ((7 + 1) / (3 - 1))
# 16.0
```

OPERADORES DE ASIGNACIÓN AUMENTADA

Operador	Equivalencia
var += 1	var = var + 1
var -= 1	var = var - 1
var *= 1	var = var * 1
var /= 1	var = var / 1
var %= 1	var = var % 1

EJERCICIO

```
>>> greeting = 'Hello'
>>> greeting += ' world!'
>>> greeting

>>> number = 1
>>> number += 1
>>> number

>>> my_list = ['item']
>>> my_list *= 3
>>> my_list
```

SOLUCIÓN

```
>>> greeting = 'Hello'
>>> greeting += ' world!'
>>> greeting
# 'Hello world!'

>>> number = 1
>>> number += 1
>>> number
# 2

>>> my_list = ['item']
>>> my_list *= 3
>>> my_list
# ['item', 'item', 'item']
```

TIPOS DE DATOS SIMPLES

Tipo de datos	Ejemplos
Números enteros (int)	-2, -1, 0, 1, 2, 3, 4, 5
Números en coma flotante (float)	-1.25, -1.0, --0.5, 0.0, 0.5, 1.0, 1.25
Cadenas de texto (str)*	'a', 'aa', 'aaa', hola!', '11 gatos', "hola 'amigo' mío"
Booleanos (bool)	True, False
Nulos	None

*Usaremos comillas dobles solamente cuando necesitemos usar comillas simples dentro de la cadena

VARIABLES

```
>>> # mal
>>> my variable = 'Hello'

>>> # bien
>>> var = 'Hello’

>>> # mal
>>> %$@variable = 'Hello'

>>> # bien
>>> my_var = 'Hello'

>>> # bien
>>> my_var_2 = 'Hello’

>>> # mal
>>> 23_var = 'hello'
```

VARIABLES

```
>>> variableCamelCase = 'No'

>>> VariablePascalCase = 'Clases'

>>> varible-kebab-case = 'No'

>>> variable_snake_case = 'Si'
```

COMENTARIOS

```
# comentario de una línea

variable = 1 # comentario de código

"""
comentario
de
varias
lineas
"""
```

FUNCIÓN PRINT()

```
>>> print('Hello world!')
# Hello world!

>>> a = 1
>>> print('Hello world!', a)
# Hello world! 1

phrase = ['printed', 'with', 'a', 'dash',\
'in', 'between']
>>> for word in phrase:
...     print(word, end='-')
...
# printed-with-a-dash-in-between-

print('cats', 'dogs', 'mice', sep=',')
# cats,dogs,mice
```

FUNCIÓN INPUT()

```
>>> print('What is your name?')
>>> my_name = input()
>>> print('Hi, {}'.format(my_name))
# What is your name?
# Martha
# Hi, Martha

>>> my_name = input('What is your name? ')
>>> print('Hi, {}'.format(my_name))
# What is your name? Martha
# Hi, Martha

>>> my_name = input('What is your name? ')
>>> print(f'Hi, {my_name}')
# What is your name? Martha
# Hi, Martha
```

FUNCIÓN LEN()

```
>>> len('hello')
# 5

>>> len(['cat', 3, 'dog'])
# 3
```

CONVERSIÓN DE TIPOS DE DATOS

```
>>> str(29)
# '29'

>>> str(-3.14)
# '-3.14'

>>> int('11')
# 11

>>> float('3.14')
# 3.14
```

OPERADORES DE COMPARACIÓN

Operador	Significado
==	Igual a
!=	No es igual a
<	Menos de
>	Mayor que
<=	Menor o igual a
>=	Mayor o igual que

EJERCICIO

```
>>> 42 == 42

>>> 40 == 42

>>> 'hello' == 'hello'

>>> 'hello' == 'Hello'

>>> 'dog' != 'cat'

>>> 42 == 42.0

>>> 42 == '42'
```

SOLUCIÓN

```
>>> 42 == 42
True

>>> 40 == 42
False

>>> 'hello' == 'hello'
True

>>> 'hello' == 'Hello'
False

>>> 'dog' != 'cat'
True

>>> 42 == 42.0
True

>>> 42 == '42'
False
```

OPERADORES BOOLEANOS

Operador	Descripción	Representación
and	True si ambos son True	x and y
or	True si al menos uno es True	x or y
not	True solo si False	not x

EJERCICIO

```
>>> (4 < 5) and (5 < 6)

>>> (4 < 5) and (9 < 6)

>>> (1 == 2) or (2 == 2)

>>> 2 + 2 == 4 and not 2 + 2 == 5 \
and 2 * 2 == 2 + 2
```

SOLUCIÓN

```
>>> (4 < 5) and (5 < 6)
True

>>> (4 < 5) and (9 < 6)
False

>>> (1 == 2) or (2 == 2)
True

>>> 2 + 2 == 4 and not 2 + 2 == 5 \
and 2 * 2 == 2 + 2
True
```

IF

```
>>> name = 'Debora'

>>> if name == 'Debora':
...     print('Hi, Debora')
...
# Hi, Debora

>>> if name != 'George':
...     print('You are not George')
...
# You are not George
```

IF

```
>>> name = 'Debora'

>>> if name == 'George':
...     print('Hi, George.')
... else:
...     print('You are not George')
...
# You are not George
```

IF

```
>>> name = 'Antony'

>>> if name == 'Debora':
...     print('Hi Debora!')
... elif name == 'George':
...     print('Hi George!')
... else:
...     print('Who are you?')
...
# Who are you?
```

WHILE

```
>>> spam = 0
>>> while spam < 5:
...     print('Hello, world.')
...     spam = spam + 1
...
# Hello, world.
# Hello, world.
# Hello, world.
# Hello, world.
# Hello, world.
```

WHILE

```
>>> while True:
...     name = input('Please type your name: ')
...     if name == 'your name':
...         break
...
>>> print('Thank you!')
# Please type your name: your name
# Thank you!
```

WHILE

```
>>> while True:
...     name = input('Who are you? ')
...     if name != 'Joe':
...         continue
...     password = input('Password? (It is a fish.): ')
...     if password == 'swordfish':
...         break
...
>>> print('Access granted.')
# Who are you? Charles
# Who are you? Debora
# Who are you? Joe
# Password? (It is a fish.): swordfish
# Access granted.
```

FOR

```
>>> pets = ['Bella', 'Milo', 'Loki']
>>> for pet in pets:
...     print(pet)
...
# Bella
# Milo
# Loki
```

FOR

```
>>> for i in range(5):
...     print(f'Will stop at 5! or 4? ({i})')
...
# Will stop at 5! or 4? (0)
# Will stop at 5! or 4? (1)
# Will stop at 5! or 4? (2)
# Will stop at 5! or 4? (3)
# Will stop at 5! or 4? (4)
```

FOR

```
# range(start, stop, step)
>>> for i in range(0, 10, 2):
...     print(i)
...
# 0
# 2
# 4
# 6
# 8

>>> for i in range(5, -1, -1):
...      print(i)
...
# 5
# 4
# 3
# 2
# 1
# 0
```

STRINGS

```
>>> a = '''Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua.'''

>>> print(a[1]) # o

>>> print(len(a)) # 123

>>> for x in "banana":
...       print(x)

>>> print('dolor' in txt) # True

>>> print('dolor' not in txt) # False
```

FUNCIONES Y ARGUMENTOS

```
>>> def say_hello(name):
...     print(f'Hello {name}')
...
>>> say_hello('Carlos')
# Hello Carlos

>>> say_hello('Wanda')
# Hello Wanda

>>> say_hello('Rose')
# Hello Rose
```

FUNCIONES Y ARGUMENTOS

```
>>> def say_hi(name, greeting):
...     print(f"{greeting} {name}")
...
>>> # without keyword arguments
>>> say_hi('John', 'Hello')
# Hello John

>>> # with keyword arguments
>>> say_hi(name='Anna', greeting='Hi')
# Hi Anna
```

FUNCIONES Y ARGUMENTOS

```
>>> def say_hi(name = 'anónimo'):
...     print(f'Hi {name}')
...

>>> say_hi('John')
Hi John

>>> say_hi()
Hi anónimo
```

FUNCIONES Y ARGUMENTOS

```
>>> def sum_two_numbers(number_1, number_2):
...     return number_1 + number_2
...
>>> result = sum_two_numbers(7, 8)
>>> print(result)
# 15
```

FUNCIONES RECURSIVAS

```
a = 0
b = 1
n=int(input("Cuántos términos?: "))
print(a,b,end=" ")
while(n-2):
    c=a+b
    a,b = b,c
    print(c,end=" ")
    n=n-1
```

FUNCIONES RECURSIVAS

```
def fibo(n):
    if n <= 1:
        return n
    else:
        return(fibo(n-1) + fibo(n-2))
```

MODULOS

```
# mymodule.py

def greeting(name):
  print("Hello, " + name)
```

MODULOS

```
>>> import mymodule
>>> mymodule.greeting("Jonathan")

>>> import mymodule as my
>>> my.greeting("Jonathan")

>>> from mymodule import greeting
>>> greting("Jonathan")
```

VARIABLES GLOBALES

- Las variables que se crean fuera de una función (como en todos los ejemplos anteriores) se conocen como variables globales.
- Las variables globales pueden ser utilizadas tanto dentro de las funciones como fuera de ellas.
- Pero si creamos una variable dentro de una función, esta variable será local, y sólo podrá ser utilizada dentro de la función.
- Para crear una variable global dentro de una función, puede utilizar la palabra clave *global*.

VARIABLES GLOBALES

```
x = "awesome"

def myfunc():
  x = "fantastic"
  print("Python is " + x)

myfunc()

print("Python is " + x)

# Python is fantastic
# Python is awesome
```

VARIABLES GLOBALES

```
x = "awesome"

def myfunc():
  global x
  x = "fantastic"

myfunc()

print("Python is " + x)

# Python is fantastic
```

TIPOS DE DATOS COMPLEJOS

- **Lista** es una colección que ordenada que se puede modificar. Permite duplicados.
  mylist = ['apple', 'banana', 'cherry']

- **Tupla** es una colección ordenada e inmutable. Permite duplicados.
  mytuple = ('apple', 'banana', 'cherry')

- **Conjunto** es una colección que es desordenada, inmutable y sin índice. No hay duplicados.
  myset = {'apple', 'banana', 'cherry'}

- **Diccionario** es una colección ordenada y que se puede modificar. No claves duplicadas (pero sí valores).
  thisdict = {'brand': 'Ford', 'model': 'Mustang', 'year': 1964}

LISTAS (EJERCICIO)

```
>>> furniture = ['table', 'chair', 'rack',
'shelf']

>>> furniture[0]

>>> furniture[-1]

>>> furniture[0:4]

>>> furniture[1:3]

>>> furniture[0:-1]

>>> furniture[:2]

>>> furniture[1:]

>>> furniture[:]
```

SOLUCIÓN

```
>>> furniture = ['table', 'chair', 'rack', 'shelf']

>>> furniture[0]
# 'table'

>>> furniture[-1]
# 'shelf'

>>> furniture[0:4]
# ['table', 'chair', 'rack', 'shelf']

>>> furniture[1:3]
# ['chair', 'rack']

>>> furniture[0:-1]
# ['table', 'chair', 'rack']

>>> furniture[:2]
# ['table', 'chair']

>>> furniture[1:]
# ['chair', 'rack', 'shelf']

>>> furniture[:]
# ['table', 'chair', 'rack', 'shelf']
```

LISTAS - FUNCIONES

En una lista l:

- len(l) → devuelve la longitud de l
- l.index(e) → devuelve el índice de e
- l.append(e) → añade e al final de l
- l.insert(p, e) → inserta e en la posición p
- del l[p] → elimina el elemento en la posición p de l
- l.remove(e) → elimina el elemento e de l (si se repite, sólo elimina el primero)
- l.pop() → elimina y devuelve el último elemento de l
- l.sort() → ordena una lista

TUPLAS

```
>>> furniture = ('table', 'chair', 'rack', 'shelf')

>>> furniture[0]
# 'table'

>>> furniture[1:3]
# ('chair', 'rack')

>>> len(furniture)
# 4

>>> tuple(['cat', 'dog', 5])
# ('cat', 'dog', 5)

>>> list(('cat', 'dog', 5))
# ['cat', 'dog', 5]

>>> list('hello')
# ['h', 'e', 'l', 'l', 'o']
```

DICCIONARIOS

```
>>> my_cat = {
...   'size': 'fat',
...   'color': 'gray',
...   'disposition': 'loud',
... }

>>> my_cat['age_years'] = 2
>>> print(my_cat)
...
# {'size': 'fat', 'color': 'gray', 'disposition':
'loud', 'age_years': 2}

>>> print(my_cat['size'])
...
# fat

>>> print(my_cat['eye_color'])
# Traceback (most recent call last):
#   File "<stdin>", line 1, in <module>
# KeyError: 'eye_color'
```

DICCIONARIOS

```
>>> pet = {'color': 'red', 'age': 42}
>>> for key in pet:
...     print(key)
...
# color
# age

>>> for key in pet.keys():
...     print(key)
...
# color
# age

>>> for item in pet.items():
...     print(item)
...
# ('color', 'red')
# ('age', 42)

>>> for key, value in pet.items():
...     print(f'Key: {key} Value: {value}')
...
# Key: color Value: red
# Key: age Value: 42
```

Y PARA ACABAR…

- El nombre Python no tiene nada que ver con reptiles, sino que procede del programa de la BBC "Monty Python's Flying Circus". Como sugiere la guía oficial de Python:

Hacer referencias a los sketches de Monty Python en la documentación no sólo está permitido, ¡sino que se fomenta!
(Making references to Monty Python skits in documentation is not only allowed, it is encouraged!)

- Esto ha llevado a los desarrolladores a incluir varias referencias ocultas no sólo en la documentación oficial, sino también en su código. Por ejemplo, en Python, las variables metasintácticas toman los nombres de *spam* y *egg*, en lugar de los más tradicionales *foo* y *bar*. Se trata de una clara referencia al sketch Spam de los Monty Python. Esto se puede ver, por ejemplo, en la documentación oficial (input y output) en la que hay muchas otras referencias como:

```
1.  >>> print 'We are the {} who say "{}!"'.format('knights', 'Ni')
2.  We are the knights who say "Ni!"
```

Y PARA ACABAR…

- Mira este sketch de Monty Python, el cual dio origen al término informático Spam:

https://es.wikipedia.org/wiki/Spam_(alimento)

https://www.youtube.com/watch?v=mG8PcUiHTpY

- Prueba estos Easter Eggs:

```
>>> import this
```

```
>>> import antigravity
```